



Universidad de las Fuerzas Armadas ESPE

Departamento de Ciencias de la Computación

Carrera de Ingeniería de Software

Aplicaciones Distribuidas

Laboratorio 2

Profesor: Geovanny Cudco

1. Tema

Desarrollo de un sistema de mensajería privada en tiempo real con Flask, SocketIO y características de privacidad avanzada (mensajes temporales y confirmación de lectura).

2. Objetivos

Al finalizar el taller, el estudiante será capaz de:

- **Comprender** la arquitectura cliente-servidor en tiempo real mediante WebSockets y el protocolo de eventos de SocketIO.
- **Implementar** un sistema de confirmación de lectura (*read receipts*) que notifique al emisor cuando el receptor visualiza un mensaje.
- **Desarrollar** una funcionalidad de mensajes temporales (*disappearing messages*) que elimine automáticamente el contenido del cliente y del servidor tras un tiempo configurable.
- **Aplicar** principios básicos de privacidad digital: no persistencia de mensajes en servidor, anonimización de metadatos y gestión segura de sesiones.
- **Diseñar** una interfaz de usuario mínima pero funcional que simule la experiencia de aplicaciones como Signal.

3. Contexto

En la actualidad, la privacidad en las comunicaciones digitales es un requisito fundamental. Aplicaciones como Signal han establecido estándares en cuanto a mensajería segura, ofreciendo funcionalidades como los mensajes temporales (desaparición automática) y las confirmaciones de lectura.

Este taller parte de un código base funcional de chat en tiempo real construido con Flask y Flask-SocketIO, y plantea el desafío de transformarlo en un sistema de mensajería privada que incorpore estas características críticas de privacidad, sin depender de bases de datos persistentes para el contenido de los mensajes.

4. Descripción

El estudiante partirá del código base proporcionado, el cual implementa un chat grupal básico con WebSockets. El trabajo consiste en extender este sistema para convertirlo en una **aplicación de chat privado tipo Signal** con las siguientes capacidades obligatorias:

4.1. Confirmación de Lectura (*Read Receipts*)

- Cuando un receptor visualiza un mensaje en su pantalla, el cliente debe emitir un evento `message_read` que informe al servidor.
- El servidor debe retransmitir esta confirmación únicamente al emisor original del mensaje.
- La interfaz debe mostrar un indicador visual (por ejemplo, doble check) que cambie de estado cuando el mensaje sea leído.

4.2. Mensajes Temporales (*Disappearing Messages*)

- El emisor podrá definir un tiempo de vida (TTL) para cada mensaje: 10 segundos, 1 minuto o 5 minutos.
- El mensaje debe visualizarse en el cliente receptor, pero iniciar una cuenta regresiva local para su eliminación automática de la interfaz.
- El servidor **no debe almacenar** el mensaje en disco ni en base de datos; solo actúa como intermediario en tiempo real.
- Opcionalmente, el servidor puede invalidar el mensaje en memoria tras el TTL para evitar retransmisión a usuarios que se conecten tarde.

4.3. Privacidad y Seguridad Básica

- El sistema debe operar en modo **privado**: no se permite el ingreso sin un nombre de usuario (pseudónimo) y un código de sala (*room*) o identificador de conversación.
- Los mensajes solo se retransmiten entre clientes que compartan la misma sala.

- No se debe mantener historial de mensajes en el servidor; la memoria del servidor solo almacena información de sesión activa.

4.4. Interfaz de Usuario Mínima

- Pantalla de ingreso: nombre de usuario y nombre de la sala privada.
- Vista de chat: burbujas de mensajes con remitente, *timestamp*, indicador de lectura y cuenta regresiva visible (para mensajes temporales).
- Lista de usuarios conectados en la sala actual.

5. Formato de Entrega

El estudiante debe entregar un enlace a un repositorio Git con su respectivo README.md

Contenido obligatorio del README.md

1. Nombre del estudiante y fecha.
2. Descripción de las funcionalidades implementadas.
3. Instrucciones paso a paso para ejecutar la aplicación.
4. Explicación técnica de cómo se implementaron los mensajes temporales y las confirmaciones de lectura.
5. Capturas de pantalla o diagrama de flujo de los eventos SocketIO utilizados.